Built-in Template Automation Test Suite Requirements

Document Control

- Version: 1.0
- Date: 2026-02-14
- Scope: Local automated validation suite for all built-in templates in DistroNexus
- Out of scope: CI/CD pipeline integration, cloud-hosted execution, template implementation changes

1. Background

DistroNexus already includes expanded built-in templates and partial automated verification. However, most runtime installation checks are still executed as manual E2E.

The project now requires a local-first, WSL2-targeted automation suite that performs real template validation, supports selective runs for newly added templates, and persists evidence to documentation artifacts.

2. Goals

- Run real validation for all built-in templates in local development WSL2 environments.
- Support selective execution for one or many templates after incremental template changes.
- Produce deterministic, archivable test evidence and summary outputs under project docs.
- Keep the suite disabled by default in CI/CD and explicitly developer-invoked.

3. Non-Goals

- No mandatory execution in pull request or mainline CI workflows.
- No replacement of existing unit/integration tests for service logic.
- No cross-platform support beyond Windows host + WSL2.

4. External Evidence Summary

- Pester supports structured advanced configuration (`New-PesterConfiguration`) with:
  - Filter by tags (`Filter.Tag`, `Filter.ExcludeTag`) for selective test execution.
  - Structured test result output (`TestResult.OutputFormat`, `TestResult.OutputPath`).
- Pester supports XML formats (`NUnitXml`, `JUnitXml`) suitable for machine parsing and report generation.
- WSL official command set provides stable host-side orchestration primitives (`wsl --list --verbose`, `wsl --distribution`, `wsl --shutdown`, `wsl --status`).
- WSL systemd guidance confirms systemd-dependent scenarios must be explicitly detected and validated.

5. Operating Constraints

- Execution environment MUST be local developer machines only.
- Execution environment MUST be Windows 10/11 host with WSL2.
- The suite MUST NOT run automatically in CI/CD by default.

- The suite MUST run against real WSL instances and real package/runtime installations (no mocked runtime validation).

6. Test Suite Architecture Requirements

6.1 Execution Layers

The suite SHALL use hybrid automation with three layers:

1. **Host Orchestration Layer (PowerShell/Pester)**
   - Selects templates and target distro(s).
   - Invokes template application flow.
   - Coordinates probe execution and report aggregation.
2. **Template Apply Layer (existing DistroNexus flow)**
   - Uses existing template application logic to perform actual installs.
3. **Runtime Probe Layer (inside WSL distro)**
   - Runs command probes to verify installed tools, versions/channels, and service availability.

6.2 Source of Truth

- The test suite SHALL discover template metadata from `config/templates.json`.
- The suite SHALL support automatic coverage of all built-in templates defined in metadata.

7. Functional Requirements

7.1 Full-Catalog Validation

- Provide an `AllTemplates` mode that executes validation for every built-in template currently present in metadata.
- For each template run, perform:
  - preflight capability checks,
  - template apply operation,
  - required probe command set,
  - structured pass/fail evaluation.

7.2 Selective Validation

- Provide a `SelectedTemplates` mode that accepts one or many template IDs.
- Input forms SHALL support:
  - comma-separated template IDs,
  - repeated parameter form (array-like invocation).
- If unknown template IDs are provided, execution MUST fail fast with clear diagnostics listing unresolved IDs.

7.3 Real Verification Rules

- Verification MUST use command execution in target WSL distro for runtime truth.
- Language templates MUST assert version/channel outcomes via tool-specific probes (for example `dotnet`, `node`, `python`, `java`, `rustc`, `go`).
- Scenario templates MUST assert functional probes (container/k8s/database/AI profile baseline checks).

- Any capability-gated scenario (GPU/systemd/Docker Desktop integration) MUST produce `Blocked` (not `Failed`) when host capability is missing.

7.4 Local-Only Execution Guard

- Runner MUST include an explicit local-only guard.
- When CI indicators are detected (for example common `CI` environment variable), default behavior SHALL skip suite execution unless an explicit override switch is passed.

8. Result and Documentation Requirements

8.1 Artifact Output

- Each run SHALL produce:
  - machine-readable test result XML (`NUnitXml` or `JUnitXml`),
  - run manifest JSON (template, distro, options, timestamps, duration, status),
  - probe output log per test case.

8.2 Documentation Output

- Each run SHALL generate a markdown summary under docs (suggested path):
  - `docs/development/testing/results/<yyyymmdd>/<run-id>/summary.md`
- Summary MUST include:
  - total templates executed,
  - pass/fail/blocked counts,
  - failed/blocked item list with reasons,
  - links to detailed logs/artifacts,
  - environment snapshot (`wsl --status`, distro, WSL version, timestamp).

8.3 Historical Traceability

- A roll-up index file SHALL be maintained (suggested: `docs/development/testing/results/index.md`) to link historical runs.

9. CLI and Usability Requirements

- Runner SHALL provide a stable command surface, for example:
  - run all templates,
  - run selected templates,
  - select distro,
  - set output directory,
  - enable/disable capability-gated scenarios.
- Runner SHALL support dry-run discovery mode to print planned execution without applying templates.

10. Acceptance Criteria

A release of this suite is acceptable when:

- All built-in templates can be executed in `AllTemplates` mode on a prepared local WSL2 environment.
- Selective mode successfully runs one template and multiple templates by ID.

- Result artifacts and markdown summary are produced for every run.
- Capability-gated scenarios are correctly classified as `Blocked` with explicit reason when prerequisites are missing.
- CI/CD default paths do not auto-run this suite.

11. Recommended Phased Delivery

- Milestone A: Runner command contract + metadata-driven test discovery.
- Milestone B: Template-family probe libraries and status classification (`Pass/Fail/Blocked`).
- Milestone C: Result persistence pipeline (XML + JSON + markdown summary + index).
- Milestone D: Full local matrix validation on Ubuntu LTS and Debian stable.

12. Risks and Mitigations

- Environment drift across developer machines:
  - Mitigate with environment snapshot and explicit capability classification.
- Long execution time for full catalog runs:
  - Mitigate with selective mode and template subset targeting.
- External package source instability:
  - Mitigate with retry policy, failure reason normalization, and rerun support.

13. References

- https://pester.dev/docs/usage/configuration
- https://pester.dev/docs/commands/New-PesterConfiguration
- https://pester.dev/docs/commands/Invoke-Pester
- https://pester.dev/docs/usage/test-results
- https://learn.microsoft.com/windows/wsl/basic-commands
- https://learn.microsoft.com/windows/wsl/systemd